Task 1 : Complex Numbers with Operator Overloading

1. Goal

In this quiz you will practice **structs** and **operator overloading with free functions**. You will design a `Complex` number type yourself, then overload the operators that make complex numbers behave like built-in numbers. Read each section, then write the declaration and definition of the requested operator before moving to the next one.

2. What is a complex number?

A complex number is written `a + bi`, where `a` and `b` are ordinary real numbers (they can have decimals, and they can be negative or zero) and `i` is a special symbol with the property `i × i = -1`.

- `a` is the **real part** (re).
- `b` is the **imaginary part** (im). It is the number that multiplies `i`.

Examples:

Complex number	Real part	Imaginary part
3 + 4i	3	4
1 - 2i	1	-2
-3 - 4i	-3	-4
5 + 0i	5	0
2.5 - 1.5i	2.5	-1.5

Two complex numbers are the same number only when both their real parts and their imaginary parts are the same.

3. Your design task

Before any operator, create the type yourself.

- Name it `Complex`, because the declarations below use that name.
- Decide what members it needs so that it can represent any complex number as described in section 2.
- Use a `struct`. No member functions: all operations are **free functions**.

4. The operations

For every operation below, `x = a + bi` is the left operand and `y = c + di` is the right operand.

4.1 Addition

To add two complex numbers, add the real parts together and add the imaginary parts together:

`(a + bi) + (c + di) = (a + c) + (b + d)i`

Example: `(3 + 4i) + (1 - 2i) = 4 + 2i`

Task: overload the arithmetic `+` operator.

```
Complex operator+(const Complex& lhs, const Complex& rhs);
```

4.2 Subtraction

Subtract the real parts and subtract the imaginary parts:

`(a + bi) - (c + di) = (a - c) + (b - d)i`

Example: `(3 + 4i) - (1 - 2i) = 2 + 6i`

Task: overload the binary `-` operator.

```
Complex operator-(const Complex& lhs, const Complex& rhs);
```

4.3 Multiplication

Multiply the two numbers like two brackets, using the distributive law, and remember that $i \times i = -1$:

$(a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i$

The term bdi^2 becomes $-bd$ because $i^2 = -1$, so it joins the real part.

Example: $(3 + 4i)(1 - 2i) = (3 + 8) + (-6 + 4)i = 11 - 2i$

Task: overload the `*` operator.

```
Complex operator*(const Complex& lhs, const Complex& rhs);
```

4.4 Division

Dividing by a complex number leaves `i` in the denominator, which is not allowed in a clean answer. To remove it, use the **conjugate**. The conjugate of `c + di` is `c - di` (same real part, opposite sign for the imaginary part).

Steps to compute `(a + bi) / (c + di)`:

1. Multiply the numerator **and** the denominator by the conjugate of the denominator, `c - di`. This does not change the value, since you multiply by `(c - di)/(c - di) = 1`.
2. Multiply out the numerator with the rule from 4.3: `(a + bi)(c - di) = (ac + bd) + (bc - ad)i`.
3. Multiply out the denominator: `(c + di)(c - di) = c² + d²`. This is a real number, so `i` is gone.
4. Divide both parts of the numerator by that real number.

Result:

`(a + bi) / (c + di) = (ac + bd)/(c² + d²) + ((bc - ad)/(c² + d²))i`

Example: `(3 + 4i) / (1 - 2i)`. Multiply top and bottom by `1 + 2i`. The numerator is `-5 + 10i`, the denominator is `1 + 4 = 5`, so the answer is `-1 + 2i`.

Task: overload the `/` operator. (Dividing by `0 + 0i` is not defined; see the bonus section.)

```
Complex operator/(const Complex& lhs, const Complex& rhs);
```

4.5 Negation

The negative of a complex number has both parts negated: `-(a + bi) = -a - bi`.

Example: `-(3 + 4i) = -3 - 4i`

Task: overload the **unary** `-` operator. It takes one operand, which is how the compiler tells it apart from subtraction.

```
Complex operator-(const Complex& c);
```

4.6 Compound assignment: `+=` and `-=`

`x += y` means "add `y` to `x` and store the result **in `x`**". Unlike `+`, this operator changes its left operand, so the left operand cannot be `const`. It returns a reference to the modified left operand so the result can be used in a larger expression.

Example: if `x = 3 + 4i` and `y = 1 - 2i`, then after `x += y` the value of `x` is `4 + 2i`. After a following `x -= y`, `x` is `3 + 4i` again.

Task: overload `+=` and `-=`.

```
Complex& operator+=(Complex& lhs, const Complex& rhs);
Complex& operator-=(Complex& lhs, const Complex& rhs);
```

4.7 Increment and decrement: `++` and `--`

In this quiz, `++` adds **1 to the real part** and leaves the imaginary part unchanged. `--` subtracts 1 from the real part. For example, `3 + 4i` becomes `4 + 4i` after `++`. This is a convention chosen for this exercise.

Each operator exists in two forms:

- **Prefix** (`++x`): change `x` first, then return `x` itself (a reference).
- **Postfix** (`x++`): remember the old value, change `x`, then return the **old value** (a copy, by value).

The postfix version has an extra unnamed `int` parameter. It is only a marker that tells the compiler "this is the postfix form". You never use its value.

Example: with `x = 1 + 1i`, the statement `cout << x++ << " " << x;` prints the old value first and the new value second.

Task: overload all four operators.

```
Complex& operator++(Complex& c);       // prefix  ++c
Complex  operator++(Complex& c, int);  // postfix c++
Complex& operator--(Complex& c);       // prefix  --c
Complex  operator--(Complex& c, int);  // postfix c--
```

4.8 Comparison: `==` and `!=`

Two complex numbers are equal when their real parts are equal **and** their imaginary parts are equal. They are different otherwise.

The two operators are opposites of each other. So `!=` must **not** repeat the comparison logic: it must call `==` and invert the result.

Task: overload `==` and `!=`.

```
bool operator==(const Complex& lhs, const Complex& rhs);
bool operator!=(const Complex& lhs, const Complex& rhs);
```

4.9 Output: `operator<<`

Printing a complex number to a stream must give the usual mathematical form `real + imagi`.

- When the imaginary part is zero or positive, print `real + imagi`.
- When the imaginary part is negative, print `real - imagi`, where the number after the minus sign is the **absolute value** of the imaginary part (so never `1 + -2i`).
- Do not print a newline or `endl` inside the function.

- Write only to the stream you receive as a parameter. Never use `cout` inside the function, because the function must also work with files and string streams.
- Return the stream by reference so that `cout << a << " " << b` works.

Expected output:

Value	Printed
3 + 4i	`3 + 4i`
1 - 2i	`1 - 2i`
-3 - 4i	`-3 - 4i`
5 + 0i	`5 + 0i`
0 + 0i	`0 + 0i`
2.5 - 1.5i	`2.5 - 1.5i`

Task: overload `<<` for `std::ostream`. It must be a free function because the left operand is the stream, not a `Complex`.

```
std::ostream& operator<<(std::ostream& os, const Complex& c);
```

4.10 Input: `operator>>`

Read two numbers from the stream: first the real part, then the imaginary part, separated by whitespace. Store them in the complex number you received and return the stream by reference.

Example: the input `7 -3` gives the complex number `7 - 3i`.

Task: overload `>>` for `std::istream`.

```
std::istream& operator>>(std::istream& is, Complex& c);
```

Both stream prototypes need `#include <iostream>` in your header.

5. Rules

1. Use **free functions only**. Do not add member functions to the struct.
2. Do not change any prototype given in this document.
3. Operators that do not change their operands must take `const` references.
4. `!=` must call `==`.
5. Postfix `++` and `--` should reuse the prefix versions instead of repeating the logic.
6. Prefix versions return `Complex&`. Postfix versions return `Complex` by value.
7. Your code must compile without errors and without warnings.

8. Bonus (optional, up to +1.5)

1. **Division by zero:** handle `x / y` when `y` is `0 + 0i` (print an error message or throw an exception). *(+0.5)*
2. **Mixed operations:** add `operator+` overloads so that `x + 5` and `5 + x` work, where `5` is a plain `double`. *(+0.5)*
3. **Helper functions:** add free functions that return the conjugate of a complex number and its magnitude (`sqrt(a² + b²)`). *(+0.5)*

Explain every bonus in a comment at the top of `Complex.cpp`.

9. Concept questions

1. Why can't `operator<<` for `cout << x` be a member function of `Complex`?
2. Why does `operator<<` return `std::ostream&` instead of `void`?
3. Why is the stream passed by non-const reference but the complex number by const reference?
4. Why does the postfix `++` have an extra `int` parameter?
5. Why must prefix `++` return a reference but postfix `++` return by value?
6. What is printed by `Complex x` holding `1 + 1i` and the statement `cout << x++ << " " << x;`?
7. Which is more efficient for a struct, `++x` or `x++`, and why?
8. This version of `operator<<` has a bug. What does it print for `1 - 2i`, and how would you fix it?

```
std::ostream& operator<<(std::ostream& os, const Complex& c) {
   os << c.real << " + " << c.imag << "i";
   return os;
}
```

(Here your struct's members happen to be named `real` and `imag`.)